

bending.forks.swe@gmail.com

Riunione con Vimar SpA

Responsabile	Aldo Toffoli	**Versione**	1.2
Redattore	Lorenzo Milanese	**Tipo**	Verbale Esterno
Verificatori	*Luca Stangherlin*	**Destinatari**	*Vimar SpA*
	Aldo Toffoli		*Prof. Tullio Vardanega*
			Prof. Riccardo Cardin

Abstract dei contenuti:
Riunione virtuale con l'azienda Vimar SpA tenutasi in data 22 ottobre 2026, per discutere il capitolato C1 AURA da loro proposto.

Digitally signed by AURA Vimar UniPD <mariano.sciacco@vimar.com>
Time: 2026-10-30 09:46:38 UTC

NOTE: the certificate is self-signed;
verify it with the signer (07e32c8c4c38).

Indice

1. Registro delle versioni — 2

2. Informazioni generali — 2

2.1. Durata e luogo — 2

2.2. Partecipanti — 3

2.2.1. Interni — 3

2.2.2. Esterni — 3

3. Contenuto della riunione — 3

3.1. Ordine del giorno — 3

3.2. Riassunto — 3

3.3. Domande e risposte — 4

3.3.1. Quale linguaggio e SDK usare per l'MCP? — 4

3.3.2. Che ruolo deve avere il client MCP? — 4

3.3.3. Cosa diamo all'utente finale per collegare il suo LLM? — 4

3.3.4. Quali risorse LLM e cloud avremo? Cosa serve per la consegna? — 5

3.3.5. Come funziona l'autenticazione MyVimar in chat? — 5

3.3.6. Comandi critici: chi blocca le richieste sbagliate? — 5

3.3.7. Polling o Subscription per lo stato e le notifiche? — 6

3.3.8. Quali richieste deve saper gestire l'assistente? — 6

3.3.9. Come si usa e si condivide il kit di impianto? — 6

3.3.10. Esiste un ambiente simulato per sviluppare in parallelo? — 6

3.3.11. Test: cosa misurano gli E2E e come si gestisce la CI? — 7

3.3.12. Quali sono le priorità e cosa aspettarsi dal PoC? — 7

3.3.13. Cosa conviene studiare subito? Quali difficoltà negli anni scorsi? — 7

3.3.14. Come si svolgono SAL e incontri durante il progetto? — 8

4. Decisioni e azioni — 8

5. Conclusioni — 8

1. Registro delle versioni

Versione	Data	Autore	Descrizione
1.0	24/10/2026	Lorenzo Milanese	Prima stesura a partire dalle presentazioni e risposte fornite da Vimar
1.0	26/10/2026	Luca Stangherlin	Verifica e approvazione del documento
1.1	27/10/2026	Lorenzo Milanese	Correzione del documento secondo le indicazioni fornite dall'azienda Vimar
1.1	27/10/2026	Luca Stangherlin	Verifica e approvazione del documento
1.2	29/10/2026	Lorenzo Milanese	Correzione del documento
1.2	29/10/2026	Aldo Toffoli	Verifica e approvazione del documento

2. Informazioni generali

2.1. Durata e luogo

- **Data:** 22 ottobre 2026
- **Inizio:** 16:00
- **Fine:** 17:00
- **Modalità:** virtuale, tramite Microsoft Teams

Il seguente documento contiene il resoconto della riunione esterna del gruppo *Bending Forks* con l'azienda *Vimar SpA* relativa al capitolato C1 *AURA*.

2.2. Partecipanti

2.2.1. Interni

Nome e Cognome	Presente	Assente
Marco Beltrame	■	□
Lorenzo Milanese	■	□
Filippo Rampazzo	□	■
Nicholas Salmaso	□	■
Luca Stangherlin	■	□
Aldo Toffoli	■	□
Matteo Wu	□	■

2.2.2. Esterni

- Mariano Sciacco (*Vimar SpA*)
- Domenico Bevilacqua (*Vimar SpA*)

3. Contenuto della riunione

3.1. Ordine del giorno

1. **Domande e risposte sui temi emersi (30–40')**: trattazione e commento congiunto dei 14 temi emersi dalle domande poste dai gruppi.
2. **Q&A live (20–25')**: sessione dedicata alle domande libere dei partecipanti, sia tramite il canale Q&A di Microsoft Teams sia a voce su alzata di mano.
3. **Prossimi passi (5')**: indicazioni sulla redazione, invio e approvazione con firma digitale dei verbali esterni di riunione.

3.2. Riassunto

L'incontro online ha visto la partecipazione dei referenti aziendali Mariano Sciacco e Domenico Bevilacqua e dei diversi gruppi interessati al capitolato C1 *AURA*. L'azienda ha raccolto preventivamente le domande poste dagli 8 gruppi interessati (incluso *Bending Forks*), aggregandole in quattro temi chiave.

I referenti hanno illustrato come la soluzione richiesta sia focalizzata principalmente sul *back-end* e sullo sviluppo di un server *MCP* conforme agli standard, demandando l'interfaccia utente ai servizi *LLM* commerciali esistenti (es. Claude, ChatGPT, Microsoft Copilot). Hanno chiarito che per l'autenticazione è mandatorio un flusso OAuth2 verso *MyVimar* in cui i token di accesso non transitino mai verso il servizio di intelligenza artificiale. È stata confermata la piena libertà di scelta del linguaggio (con una preferenza suggerita per Python con FastMCP), la fornitura di un kit fisico collegato via cloud a un account amministratore e la possibilità di simulare le API tramite mock OpenAPI (es. Prism) per lavorare in parallelo senza blocchi. Infine, sono state delineate le

modalità di svolgimento degli incontri di SAL e le linee guida per la stesura e l'approvazione del presente verbale.

3.3. Domande e risposte

3.3.1. Quale linguaggio e SDK usare per l'MCP?

- **Libertà totale sul linguaggio:** Non vi sono vincoli rigidi da parte dell'azienda stakeholder; il gruppo può scegliere in autonomia tra Python, Java o TypeScript in base alle proprie competenze interne ed esperienze pregresse.
- **Python + FastMCP consigliato:** È l'SDK ufficiale suggerito all'interno del capitolato e rappresenta lo stack tecnologico su cui l'azienda proponente può offrire il supporto tecnico più mirato ed efficace.
- **Omogeneità di SDK per client e server:** L'impiego del medesimo SDK sia sul versante server che su quello client facilita notevolmente la scrittura e l'esecuzione dei test d'integrazione tra le parti.
- **Suggerimento aziendale:** Se il team possiede una solida preparazione in Java, questa opzione è ritenuta valida; tuttavia, Python resta il percorso più rapido ed efficiente per avviare lo sviluppo.

3.3.2. Che ruolo deve avere il client MCP?

- **Controllo e utilizzo da parte dell'LLM:** L'obiettivo finale è consentire al modello di linguaggio di interrogare ed eseguire le funzioni esposte dal server MCP. I client commerciali moderni (come Claude o ChatGPT) integrano già un client MCP nativo.
- **Finalità del client di progetto:** Lo sviluppo di un client ad hoc serve prevalentemente per agevolare l'esecuzione dei test d'integrazione. Eventuali interfacce CLI/TUI o estensioni per VS Code costituiscono componenti puramente opzionali; il gruppo è libero di utilizzare strumenti come Claude Code via CLI e Codex CLI.
- **Ripartizione di responsabilità ed errori:** La separazione dei compiti nella gestione degli errori (ad esempio, la mancata raggiungibilità di un attuatore) tra server e client costituisce parte integrante delle scelte architetturali demandate al gruppo.
- **Suggerimento aziendale:** Una volta implementato un server MCP solido, l'automazione avanzata non richiede necessariamente componenti dedicati, ma può essere ottenuta definendo un prompt o una skill apposita. Il focus prioritario rimane il controllo puntuale in lettura e scrittura dei dispositivi fisici tramite API.

3.3.3. Cosa diamo all'utente finale per collegare il suo LLM?

- **Assenza di frontend proprietario:** Non è richiesto lo sviluppo di un'app nativa o di un sito web; il sistema è interamente *back-end*. L'utente finale interagirà con il sistema sfruttando le interfacce conversazionali già fornite dai vari provider di LLM.
- **Ambiente locale per sviluppo e server remoto per rilascio:** Per le fasi di sviluppo, debug e test interni è consigliato adottare un server MCP locale. Per l'utente finale sarà invece necessario disporre di un server MCP remoto con URL pubblicamente raggiungibile e protetto da autenticazione, fruibile dal servizio AI via web, desktop e mobile.
- **Supporto ai servizi AI commerciali:** È sufficiente che la soluzione si interfacci con un singolo servizio commerciale a scelta; il supporto opzionale a molteplici servizi è apprezzato, ma non richiede alcuna interfaccia grafica dedicata di selezione.

- **Suggerimento aziendale:** La scelta di provider LLM differenti da parte dei diversi gruppi (Claude, ChatGPT, Copilot) consentirà all'azienda di mettere a confronto e valutare approcci architetturali alternativi.

3.3.4. Quali risorse LLM e cloud avremo? Cosa serve per la consegna?

- **Fruizione degli LLM:** Per il capitolato è possibile avvalersi delle funzionalità gratuite o destinate agli studenti offerte dalle piattaforme commerciali (ad esempio connettori personalizzati di Claude o GitHub Copilot tramite GitHub Student Developer Pack).
- **Infrastruttura AWS opzionale:** L'accesso a risorse cloud AWS è opzionale: qualora richiesto e motivato, Vimar potrà concedere l'accesso a un insieme ristretto di servizi AWS e, se necessario, una chiave API per prove con gli LLM (da concordare all'avvio del progetto).
- **Modalità di consegna del prodotto:** Ai fini della consegna e della valutazione finale è sufficiente che l'intera architettura sia eseguibile in locale tramite container Docker. Durante il collaudo conclusivo, i referenti si renderanno disponibili al supporto e alla configurazione della soluzione su AWS o su un server dedicato.

3.3.5. Come funziona l'autenticazione MyVimar in chat?

- **Profilazione e accesso agli impianti:** L'interazione richiede un account *MyVimar* valido abilitato all'accesso agli impianti. Ciascun utente deve poter visualizzare e operare unicamente sui propri impianti associati, garantendo il completo isolamento rispetto a impianti di terzi.
- **Riservatezza dei token di accesso:** Il server MCP gestisce l'associazione tra l'identità dell'utente e l'autorizzazione ottenuta via protocollo OAuth2 per dialogare con le API Vimar. Il servizio LLM esterno non deve in nessun caso venire a conoscenza dei token di autenticazione.
- **Progettazione del flusso di login:** La sequenza di passaggi attraverso cui l'utente immette le proprie credenziali (es. link di autorizzazione aperto nel browser, concessione esplicita di permessi in lettura/scrittura) costituisce oggetto di analisi e progettazione a carico del team.
- **Suggerimento aziendale:** Rispettare rigorosamente i criteri di privacy: le informazioni di stato dell'impianto possono pervenire al modello per elaborare la risposta, mentre le informazioni di sicurezza sensibili (token, segreti) non devono mai essere trasmesse all'LLM.

3.3.6. Comandi critici: chi blocca le richieste sbagliate?

- **Comportamento delle API aziendali:** Le API Vimar eseguono operazioni di lettura e scrittura agendo come quelle di una qualsiasi applicazione; non implementano meccanismi di validazione o filtri specifici per richieste originate da un LLM.
- **Richiesta di conferma per operazioni distruttive o massive:** Mentre le letture di stato possono essere eseguite direttamente, per le operazioni di scrittura ad alto impatto (es. "spegni tutte le luci dell'edificio") è opportuno prevedere un passaggio esplicito di conferma da parte dell'utente.
- **Configurazione granulare dei permessi:** Il protocollo KNX IoT permette di impostare *scope* di autorizzazione differenti (ad es. permessi di sola lettura, lettura e scrittura completa, o accesso ristretto a un sottoinsieme di dispositivi).
- **Suggerimento aziendale:** L'agente deve essere progettato in modo da interpretare correttamente il contesto della richiesta e invocare il tool appropriato. L'implementazione di guardrail o skill di sicurezza aggiuntive può essere valutata in fase di analisi dei requisiti.

3.3.7. Polling o Subscription per lo stato e le notifiche?

- **Adozione del Polling:** Come chiarito dall'azienda, l'interrogazione tramite polling periodico verso le API per la gestione dei dispositivi è considerata una soluzione pienamente accettabile e sufficiente per le finalità del progetto.
- **Integrazione via Webhook:** I servizi LLM standard di norma non dispongono di meccanismi nativi per ricevere notifiche asincrone innescate da un webhook esterno, sebbene sia possibile investigare se specifici provider offrano tale funzionalità e in caso valutarli come possibili requisiti *nice-to-have* od opzionali.
- **Inquadramento delle Subscription:** Il caso d'uso avanzato #21 ("avvisami quando si verifica un evento...") è classificato come requisito *nice-to-have* e può essere implementato sotto forma di azione programmatica.
- **Suggerimento aziendale:** I meccanismi di subscription sono indicati per eventi sporadici e non temporalmente prevedibili; si raccomanda di affrontarli unicamente a seguito del completamento e della stabilizzazione di tutti i casi base.

3.3.8. Quali richieste deve saper gestire l'assistente?

- **Flessibilità espressiva:** Gli utenti finali non formuleranno i comandi utilizzando le esatte frasi indicate negli esempi; l'assistente LLM deve essere capace di interpretare sinonimi, variazioni sintattiche e richieste parzialmente ambigue.
- **Esempi come linea guida:** I casi d'uso elementari e avanzati riportati nel capitolato rappresentano scenari indicativi e non esaustivi, fungendo da base di partenza per l'individuazione dei requisiti software.
- **Valorizzazione di nuove casistiche:** L'ideazione e la gestione di interazioni innovative non contemplate inizialmente sono considerate elementi *nice-to-have* di grande valore da discutere con i referenti.
- **Suggerimento aziendale:** Si suggerisce di utilizzare un modello di linguaggio per riformulare i casi della test list, sottoponendoli a diversi LLM per testare e accrescere la robustezza dell'assistente.

3.3.9. Come si usa e si condivide il kit di impianto?

- **Connettività su rete cloud:** Il kit fisico domotico si connette alla rete Wi-Fi locale ed è gestito interamente via cloud tramite le API KNX IoT, senza richiedere collegamenti USB fisici ai PC degli sviluppatori né configurazioni di VPN aziendali.
- **Unico nodo fisico, accesso distribuito:** È sufficiente che un unico componente del gruppo mantenga il kit alimentato e connesso alla rete domestica; tutti gli altri componenti del team possono operare contemporaneamente da remoto interfacciandosi con le API cloud.
- **Gestione degli account:** Vimar fornirà un account con privilegi di amministrazione; sarà possibile registrare e associare profili addizionali previo inserimento sul portale *MyVimar*.
- **Suggerimento aziendale:** Il modo più immediato per apprendere la struttura del modello dati consiste nel premere fisicamente un comando sul kit e analizzare le corrispondenti variazioni di stato notificate via API.

3.3.10. Esiste un ambiente simulato per sviluppare in parallelo?

- **Server mock da specifica OpenAPI:** Utilizzando il file di specifica YAML OpenAPI di KNX IoT unitamente a tool open source quali ad esempio *Prism*, è possibile istanziare in locale server

di mock capaci di restituire payload conformi e realistici.
- **Piattaforma demo online:** Nella documentazione dedicata agli integratori è resa disponibile una demo online KNX IoT provvista di API di simulazione attive e consultabili.
- **Discrepanze concettuali:** I simulatori software non replicano fedelmente tutte le peculiarità del concetto di "impianto" (*plant*) proprio dell'ecosistema Vimar; pertanto, le verifiche conclusive dovranno sempre avvenire a fronte del kit reale.
- **Suggerimento aziendale:** Impiegare i mock per le attività quotidiane di sviluppo parallelo (prevenendo conflitti e colli di bottiglia tra membri) e riservare il kit hardware alle validazioni *end-to-end*.

3.3.11. Test: cosa misurano gli E2E e come si gestisce la CI?

- **Copertura dei test E2E sui requisiti:** L'obiettivo dell'80% di copertura per i test *End-to-End* è calcolato sulla totalità dei requisiti di progetto concordati. La lista di scenari di test nel capitolato copre le interazioni base dell'utente con l'assistente AI.
- **Mock degli LLM nelle pipeline di CI:** All'interno dei test automatizzati (unitari e di integrazione) integrati nella Continuous Integration è vivamente raccomandato simulare le risposte dell'LLM tramite mock, assicurando esecuzioni deterministiche, riproducibili e prive di costi di computazione.
- **Copertura di unità e integrazione:** Per tali livelli di test fa fede la classica metrica di *code coverage*, con una soglia minima prefissata al 75% e un traguardo ottimale superiore al 90%.
- **Suggerimento aziendale:** Poiché un vero modello linguistico manifesta un comportamento intrinsecamente non deterministico, rischiando di far fallire casualmente le pipeline, se ne sconsiglia caldamente l'invocazione diretta nei job di CI.

3.3.12. Quali sono le priorità e cosa aspettarsi dal PoC?

- **Dalle esigenze di business ai requisiti formali:** Il documento di capitolato sintetizza bisogni commerciali generali; spetta interamente al gruppo di sviluppo dedurre, specificare e gerarchizzare i requisiti dettagliati durante la fase di analisi.
- **Obiettivi del Proof of Concept (PoC):** Il prototipo iniziale deve mostrare un server MCP operativo in grado di assolvere a 2–3 scenari basilari: login e autenticazione all'impianto, restituzione della lista degli impianti abbinati e lettura/modifica dello stato di un attuatore di illuminazione.
- **Valore dei requisiti nice-to-have vs opzionali:** Nessuna delle due categorie è vincolante per l'accettazione; tuttavia, un requisito di tipo *nice-to-have* ben rifinito ed efficace offre all'azienda maggior valore rispetto a un requisito opzionale convenzionale.
- **Suggerimento aziendale:** Costruire la gerarchia dei requisiti partendo rigidamente dai vincoli obbligatori e dai casi d'uso essenziali, riservando valutazioni su connettori o skill aggiuntive alle fasi successive di analisi e scouting.

3.3.13. Cosa conviene studiare subito? Quali difficoltà negli anni scorsi?

- **Tempistiche di auto-formazione:** Non vi sono vincoli temporali rigidi prima dell'assegnazione ufficiale; nondimeno, acquisire familiarità preliminare con il dominio tecnologico permette una scelta consapevole del capitolato.
- **Fonti di studio consigliate:** Si raccomanda di consultare preliminarmente le specifiche KNX IoT per integratori, le peculiarità delle primitive proprietarie Vimar e la documentazione ufficiale dell'architettura Model Context Protocol.

- **Criticità riscontrate nelle edizioni pregresse:** Le maggiori problematiche emerse in passato hanno riguardato: una comprensione approssimativa o ambigua dei requisiti nelle settimane iniziali, una documentazione interna disordinata e severi colli di bottiglia dovuti a tentativi di eseguire modelli LLM localmente.
- **Suggerimento aziendale:** Mantenere un dialogo costante con i referenti tecnici per dissipare qualunque dubbio e non optare per modelli in esecuzione locale, preferendo servizi cloud commerciali accessibili via API.

3.3.14. Come si svolgono SAL e incontri durante il progetto?

- **Modalità dei SAL:** I meeting periodici di Avanzamento Lavori (SAL) si svolgono ordinariamente da remoto tramite Microsoft Teams; qualora il team ne manifesti la preferenza, vi è disponibilità a ospitare gli incontri in presenza presso le sale riunioni della sede Vimar.
- **Incontri in presenza e dimostrazioni tecniche:** Sono richiesti due incontri obbligatori in sede: il kick-off per la consegna fisica del kit domotico e il collaudo conclusivo del sistema. Sono inoltre previste circa due sessioni dimostrative congiunte con il reparto R&D: una a metà percorso in occasione del PoC e una alla chiusura del progetto (MVP).
- **Supporto aziendale:** L'azienda assicura massima disponibilità per approfondimenti di natura architetturale, tecnica o di aderenza alle esigenze espresse, declinando invece revisioni formali sugli aspetti burocratici o accademici dei documenti di progetto.
- **Suggerimento aziendale:** Eventuali richieste di sessioni formative o incontri con specialisti di team specifici (es. esperti di sicurezza o protocolli) possono essere concordate con un preavviso di 1–2 settimane.

4. Decisioni e azioni

ID	Descrizione	Assegnatario
DEC-001	Inviare il verbale via *email* all'indirizzo da cui è arrivata la richiesta, per la firma dell'azienda	Filippo Rampazzo
DEC-002	Integrare le risposte ricevute nella valutazione dei capitolati	Team Bending Forks

5. Conclusioni

Il gruppo *Bending Forks* esprime il proprio sentito ringraziamento ai referenti di *Vimar SpA*, Mariano Sciacco e Domenico Bevilacqua, per la chiarezza espositiva, la ricchezza del materiale condiviso e la costante disponibilità dimostrata verso tutti i team. I chiarimenti tecnici e organizzativi forniti costituiscono elementi determinanti per orientare in maniera consapevole e strutturata le scelte del gruppo in vista della candidatura ufficiale ai capitolati d'appalto.